

NAME: Shaikh Areeba Mohammed Ismail

CLASS/BATCH: TE-B2

ROLL NO: 46

SUBJECT: Artificial Intelligence

GROUP: A

ASSIGNMENT NO: 04

Problem Statement: Implement a solution for a Constraint Satisfaction Problem using Branch and Bound and backtracking for n-queens problem or a graph coloring problem.

1.Graph Coloring Algorithm (Greedy Method)

```
# Adjacency Matrix Representation of Graph
# Nodes: a, b, c, d, e, f

adj_matrix = [
    [0,1,1,1,0,0],
    [1,0,1,0,1,0],
    [1,1,0,1,1,1],
    [1,0,1,0,0,1],
    [0,1,1,0,0,1],
    [0,0,1,1,1,0]
]

nodes = ['a','b','c','d','e','f']
n = len(nodes)

print ("Adjacency Matrix Loaded Successfully")

# Calculate Degree of Each Node
degree = [0]*n

for i in range(n):
    degree[i] = sum(adj_matrix[i])

print("Degree of Nodes:")
for i in range(n):
    print(nodes[i], "=", degree[i])
```

```

# Sort Nodes by Degree (Descending Order)
sorted_nodes = nodes.copy()
sorted_degree = degree.copy()

for i in range(n):
    max_idx = i
    for j in range(i+1, n):
        if sorted_degree[j] > sorted_degree[max_idx]:
            max_idx = j

    # Swap degree
    sorted_degree[i], sorted_degree[max_idx] =
sorted_degree[max_idx], sorted_degree[i]

    # Swap nodes
    sorted_nodes[i], sorted_nodes[max_idx] =
sorted_nodes[max_idx], sorted_nodes[i]

print("Nodes sorted by degree:")
print(sorted_nodes)

# Graph Coloring using Greedy Algorithm
colors = {}
available_colors = ["Red", "Blue", "Green", "Yellow", "Orange"]

for node in sorted_nodes:
    index = nodes.index(node)

    # Find colors used by adjacent nodes
    used_colors = set()
    for j in range(n):
        if adj_matrix[index][j] == 1:
            neighbor = nodes[j]
            if neighbor in colors:
                used_colors.add(colors[neighbor])

    # Assign first available color
    for color in available_colors:
        if color not in used_colors:
            colors[node] = color
            break

print("Graph Coloring Solution:")
for node in nodes:
    print("Node", node, "=", colors[node])

```

OUTPUT:

Adjacency Matrix Loaded Successfully

Degree of Nodes:

a = 3

b = 3

c = 5

d = 3

e = 3

f = 3

Nodes sorted by degree:

['c', 'b', 'a', 'd', 'e', 'f']

Graph Coloring Solution:

Node a = Green

Node b = Blue

Node c = Red

Node d = Blue

Node e = Green

Node f = Yellow

2. N - QUEENS PROBLEM

```
# Python program to solve N Queen
# Problem using backtracking
global N
N = 4
def printSolution(board):
    for i in range(N):
        for j in range(N):
            print (board[i][j],end=' ')
        print()
# A utility function to check if a queen can
# be placed on board[row][col]. Note that this
# function is called when "col" queens are
# already placed in columns from 0 to col -1.
# So we need to check only left side for
# attacking queens
def isSafe(board, row, col):
    # Check this row on left side
    for i in range(col):
        if board[row][i] == 1:
            return False
    # Check upper diagonal on left side
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False
    # Check lower diagonal on left side
    for i, j in zip(range(row, N, 1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False
    return True
def solveNQUtil(board, col):
    # base case: If all queens are placed
    # then return true
    if col >= N:
        return True
    # Consider this column and try placing
    # this queen in all rows one by one
    for i in range(N):
        if isSafe(board, i, col):
            # Place this queen in board[i][col]
            board[i][col] = 1
            # recur to place rest of the queens
            if solveNQUtil(board, col + 1) == True:
                return True
            # If placing queen in board[i][col]
            # doesn't lead to a solution, then
            # queen from board[i][col]
```

```

        board[i][col] = 0
        # if the queen can not be placed in any row in
        # this column col then return false
        return False
# This function solves the N Queen problem using
# Backtracking. It mainly uses solveNQUtil() to
# solve the problem. It returns false if queens
# cannot be placed, otherwise return true and
# placement of queens in the form of 1s.
# note that there may be more than one
# solutions, this function prints one of the
# feasible solutions.
def solveNQ():
    board = [ [0, 0, 0, 0],
               [0, 0, 0, 0],
               [0, 0, 0, 0],
               [0, 0, 0, 0]
             ]
    if solveNQUtil(board, 0) == False:
        print ("Solution does not exist")
        return False
    printSolution(board)
    return True
solveNQ()

```

OUTPUT:

```

0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
True

```